Introduction

Minsoo Ryu

Hanyang University

Course Information

❒ Teacher

- 컴퓨터소프트웨어학부 유민수
- msryu@hanyang.ac.kr
- http://osdc.hanyang.ac.kr/

❒ Textbook

- **Software Engineering, 9th Edition (Ian Sommerville, Addison-Wesley)**

❒ Grading

- **Final exam 90%, attendance 10%**

Intro: What and Why?

What is Software Engineering?

- Software engineering is an engineering discipline that is concerned with all aspects of software production
  - Software engineers should adopt a systematic and organized approach to their work and use appropriate tools and techniques depending on the problem to be solved, the development constraints and the resources available

- The term software engineering was popularized by F.L. Bauer during the NATO Software Engineering Conference in 1968
  - Software engineering was proposed to discuss 'software crisis'
  - Software crisis: orders of magnitude larger and more complex

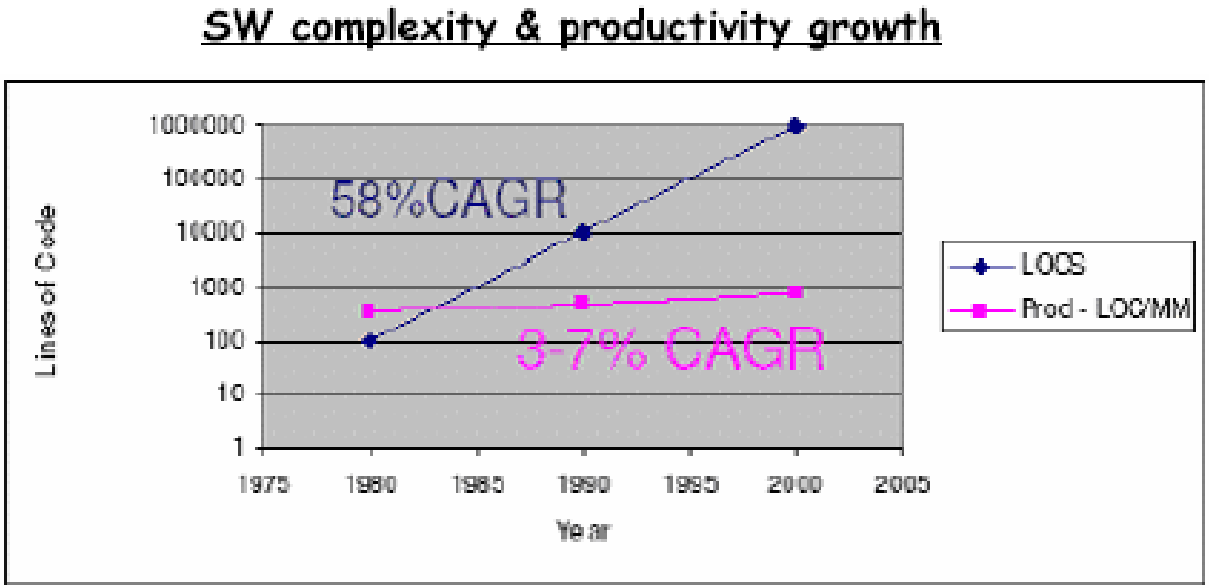
Software Crisis

- Software crisis was used to describe the impact of rapid increases in computer power and the complexity of the problems which could be tackled
  - In essence, it refers to the difficulty of writing correct, understandable, and verifiable computer programs
  - The roots of the software crisis are complexity, expectations, and change

- The software crisis manifested itself in several ways:
  - Projects running over-budget
  - Projects running over-time
  - Software was of low quality
  - Software often did not meet requirements
  - Projects were unmanageable and code difficult to maintain

Software Crisis

- Dominique Potier, Vice President at Thales, EU Information Society Technology, 2004
  - Embedded S/W **Complexity: 58% annual increase**
  - Embedded S/W **Productivity: 3% – 7% annual increase**



Embedded S/W Productivity by a single engineer is 0.5 – 5.0 LOC/Hour

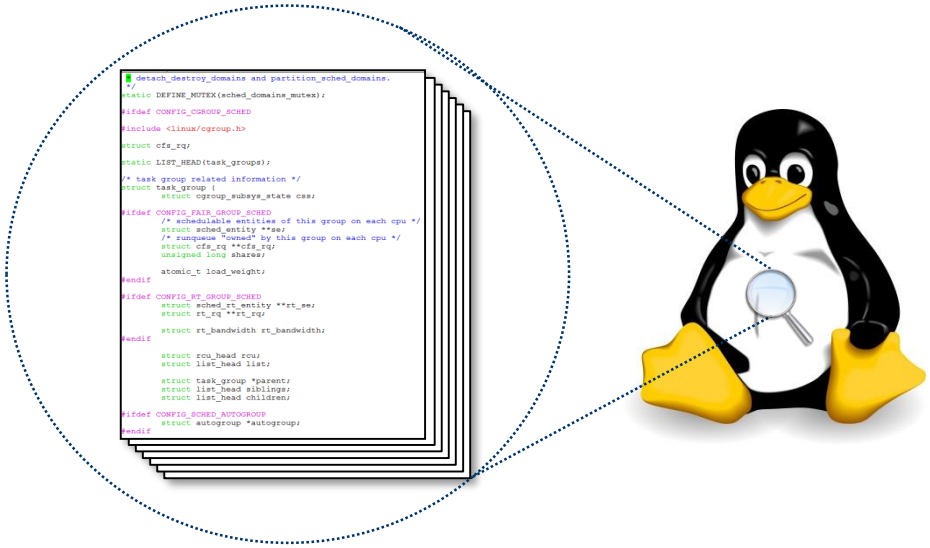
Project Size

- A big project may be 100 to 1,000+ person years
  - Most software development is by teams
  - Effectiveness of team determines success
  - Every important program is written by many people, who are constantly changing

- No large system is ever complete
  - Most large software projects are built on older ones
  - It is rare to start a new suite of programs from scratch
  - Building on the work of others is a fundamental skill of software development
  - Before a big project is completed the requirements change many times

Example: Operating Systems

- The Linux kernel has around 27.8 million lines of code in its Git repository (2020)
  - About 8,000 man-years of development effort
  - Over $1 billion (in U.S. dollars)



Linux has 27M lines of code

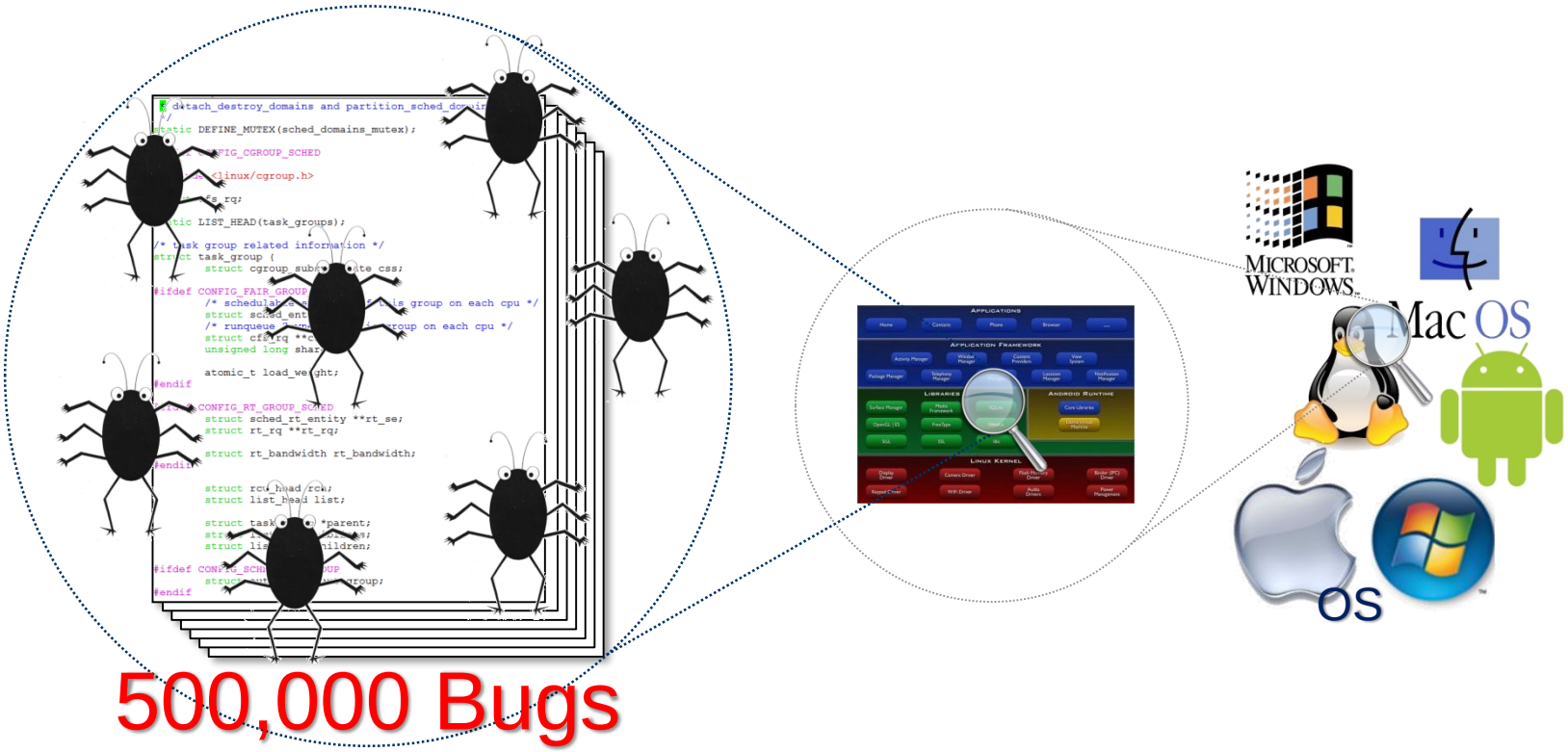


45M lines of code

Too Many Bugs

- "about 15 - 50 errors per 1000 lines of code"

  - Steve McConnell, *Code Complete*



Linux: more than 27M lines of code!

Variety of Software Products

- Real time: automotive control, air traffic control
- Consumer electronics: smartphone, washing machine
- Embedded systems: digital camera, GPS
- Data processing: telephone billing, pensions
- Information systems: web sites, digital libraries
- Sensors: weather data
- System software: operating systems, compilers
- Communications: routers, mobile telephones
- Offices: word processing, video conferences
- Scientific: simulations, weather forecasting
- Graphical: film making, design
- etc., etc., etc., ...

Management Issue: Risk

- **Problems**
  - **Over budget**
    - The major costs are salaries (your salaries)!
  - **Late delivery**
  - **Does not work as expected**

- **Never used**
  - **Does the wrong thing**
  - **Needs change**
  - **Users dislike to use it**
  - **etc.**

Management Issue: Visibility

- **Problem**
  - Must rely on others for reports of progress or difficulties
- **Software Developers**
  - Have difficulty evaluating progress
  - Optimistic
  - Consider reporting a waste time
  - etc.
- ***The people who take the responsibility must know what is happening!***

What is the Goal of Software Development?

❒ Goal of software development

- **Deliver good software under time and cost constraints**

❒ Good software?

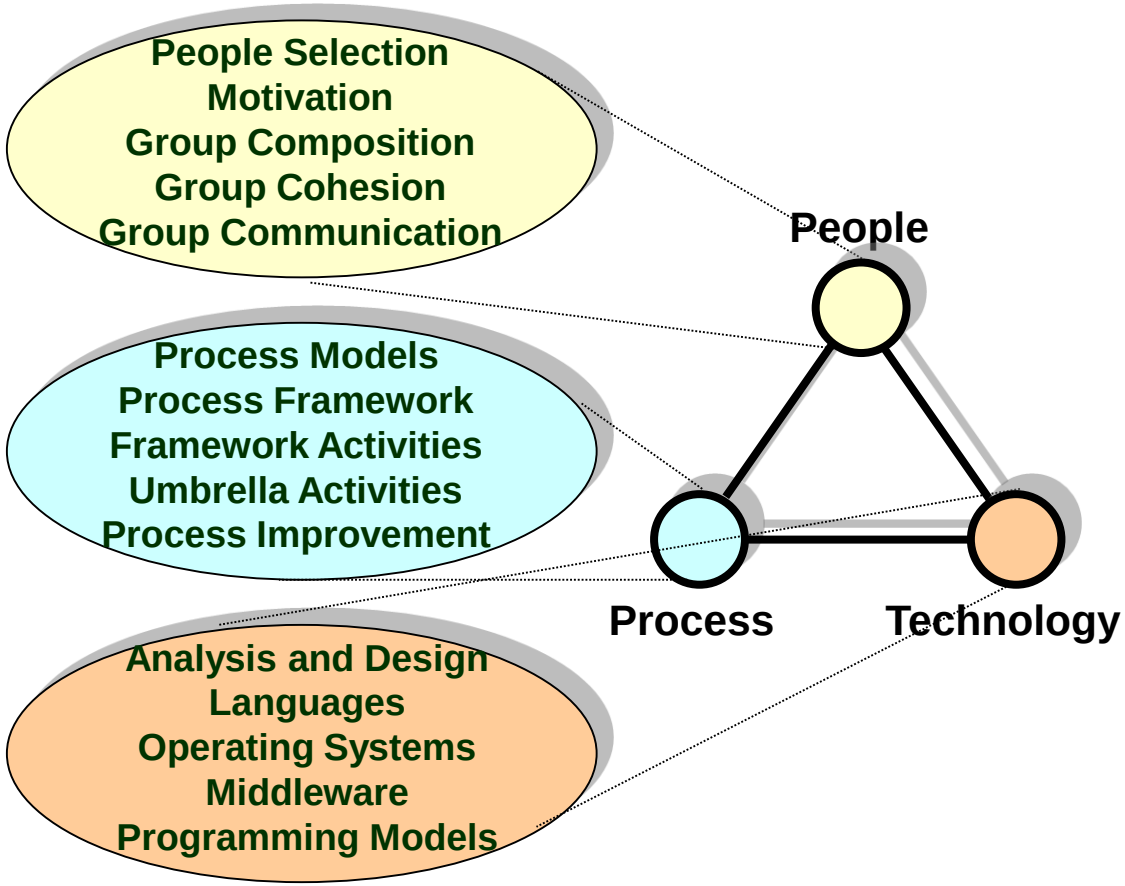
- **General characteristics**
  - Usability
  - Maintainability
  - Dependability
  - Efficiency
- **Good software should satisfy users' needs**
  - Good software products require good programming, but programming quality is the means to the end, not the end itself

Any Solution?

- Various processes and methodologies have been developed over the last few decades to "tame" the difficulty of software development, with varying degrees of success

- However, it is widely agreed that there is no "silver bullet" — that is, no single approach which will prevent project overruns and failures in all cases
  - In general, software projects which are large, complicated, poorly-specified, and involve unfamiliar aspects, are still particularly vulnerable to large, unanticipated problems

The Triad for Software Development



"People are the most important aspect of the triad because with good people, the correct processes and technology can be developed, tested and implemented."

Tools are the fourth factor that affects the success of S/W project. With good tools, people, process, and technology can be best managed for better productivity and quality


Q & A

소프트웨어 개발에 대한 고찰

- 문제(**problem**)를 정의하기 어렵다 (**wicked problem**)
  - 요구사항(**requirements**)을 정의하기 어렵다
- 개발 도중에 문제가 변경된다 (**changes are common**)
  - 요구사항(**requirements**)이 빈번하게 변경된다
- 설계(**design**)가 어렵다 (**design is communication**)
  - 많은 경험과 지식, 그리고 통찰력을 필요로 한다
  - 설계는 소통이다
    - If you can't explain it, you are not done designing it yet
- 평가(**evaluation**)가 어렵다
  - 소프트웨어의 좋고 나쁨을 객관적으로 평가하기 어렵다

소프트웨어 개발에 대한 고찰

- 최고의 프로그래머는 최하의 프로그래머보다 **28**배 뛰어나다
- 문제의 복잡성이 **25%** 증가하면 솔루션의 복잡성은 **100%** 증가한다
- 소프트웨어 작업의 **80%**가 지적인 작업이다
  - 그 중 상당 부분이 창조적인 작업이며 사무적인 일은 거의 없다.
  - 분석, 설계, 구현은 물론 문서화 또한 고도의 지적 작업이다
- 소프트웨어공학은 가르칠 수 없다
  - 소프트웨어공학은 경험에서 배운다